FlowCausal：利用工具返回关系调度并发代码智能体工作流

摘要。并发代码智能体通过不断变化的进程树执行构建、测试和代码库操作。Linux 能够在可运行任务与控制组（cgroup）之间分配 CPU 时间，却无法直接识别哪些进程必须完成，当前工具调用才能返回。复用沙箱还会保留先前工具产生的后台进程，使进程来源与当前等待依赖不再一致。

本文提出以 C/eBPF 实现的调度器 FlowCausal 。C 程序登记 cgroup、会话和工具调用标识；BPF 根据进程生命周期与实际阻塞事件确认工具返回关系。调度器先按会话分配 CPU 份额，再向已确认的等待对象提供有上限的提前运行，实际服务仍计入所属会话。等待结束、任务退出或工具调用变化时，相应关系失效；在线调度无需轮询或扫描进程树。

HarnessReplayBench 包含来自 42 个代码库的 50 个任务和 2,940 条工具命令。两组四会话实验中，每种方案交错运行五轮；FlowCausal 相对 Linux 将平均完成时间降低 4.2%–5.2%，相对 `scx_simple` 降低 27.9%–31.9%。构建等待消融表明，工具返回关系相对仅会话调度进一步降低完成时间 4.88%。在 4 个 CPU、8 个并发工作流、每种方案运行三轮的独立对照中，FlowCausal 相对原生 Linux 将平均完成时间降低 7.32%，全部完成时间增加 2.65%。跨命令依赖原型在固定工作量消融中降低平均完成时间 15.79%，但未在完整工具回放中取得额外收益。实验结果支持利用实际等待关系安排会话内的执行次序，并表明其收益取决于等待对象的计算量及依赖转换条件。

关键词： Linux 调度，代码智能体，工具执行，进程关系，`sched_ext` ，eBPF。

1 引言

代码智能体正在把代码库维护变成主机上的系统负载。本文把智能体运行时发起的一次命令、构建、测试或代码库操作称为一次“工具调用”，把一个智能体连续发起的多次工具调用称为一个“会话”。一个会话会在容器或微虚拟机中启动命令解释器、语言运行时、编译器、测试框架、软件包管理器和辅助服务。在同一主机集中运行多个会话能够提高利用率，却也使完成时间取决于主机如何在多棵不断变化的进程树之间分配处理器时间。本文把当前工具调用直接启动的进程称为“工具进程”。在实验中，每个重放的软件工程任务占用一个会话，因此“工作流完成时间”就是该会话从启动到结束的时间。

这里的调度目标不同于单个任务完成。用户看到的是当前工具调用何时返回以及智能体何时能够继续，Linux 看到的则是可运行进程。阻塞在 `wait` 中的工具进程可能等待测试进程，而这些进程又与先前工具调用遗留的计算进程竞争处理器。两者可以具有相同的控制组和优先级。因此，按任务公平分配处理器时间仍可能推迟当前工具调用必须等待的进程。图 1 给出了这种差异：在一条公开 DVC 记录中，52 个能够解除当前等待的进程与先前工具调用产生的计算同时运行。

现有主流机制提供了相邻能力。Linux 控制组在容器之间分配处理器时间；AgentCgroup 将资源核算与智能体工具调用对应 [44]；工作流调度器处理显式作业或依赖图 [22, 34, 38]；可编程调度框架提供策略接口 [13, 20]。这些机制分别工作在任务、控制组、工具或者已声明工作流的边界上。

但它们都没有告诉处理器调度器：当前有哪些存活子进程必须结束，一个已经阻塞的工具调用才能返回。父子关系也不足以回答这个问题，因为父进程可以拥有不等待的异步子进程，等待函数也可能立即返回。在用户态周期性扫描 `/proc` 不仅会晚于短进程，还无法把进入等待与紧随其后的调度状态作为同一个事实处理。本文把已阻塞工具进程与能够解除其等待的存活子进程之间的关系称为“工具返回关系”。精确问题因而是：如何确认当前工具调用的这条关系，在条件变化后及时删除，并且不破坏多个会话之间的处理器时间分配。

本文提出 FlowCausal ，并将两个决策分开。外层按照加权虚拟运行时间在会话之间分配处理器时间，避免进程较多的会话仅凭任务数量获得更多资源；内层允许工具返回关系中的一个进程在会话内普通顺序之前获得一个短时间片，所用时间仍计入所属会话。小型 C 程序只登记会话和工具调用编号；eBPF 在进程创建时传递这些编号，根据内核事件确认阻塞等待，并在等待、进程或工具调用变化后删除关系。在线系统不轮询进程，也不依赖命令名称。

本文使用由 42 个公开代码库中 50 个智能体任务构成的 HarnessReplayBench 评价系统。两组四会话实验对每种方法交错运行五轮，并与 Linux、使用同权控制组的 Linux 和上游 `scx_simple` 比较。FlowCausal 相对 Linux 将平均工作流完成时间缩短 4.2%–5.2%，相对 `scx_simple` 缩短 27.9%–31.9%，并相对 Linux 将最后一个工作流的完成时间缩短 1.7%–2.6%。使用同一份代码的消融实验表明，当构建工作决定工具返回时，依据工具返回关系安排进程次序可进一步缩短 4.88%。系统识别了工具返回关系中 99.04% 的进程，内核事件零丢失。

本文贡献如下：

- **会话级处理器调度**。把智能体会话作为长期处理器时间的分配单位，把会话中的进程作为短期执行单位。两级设计避免进程数量决定会话获得的处理器时间，同时保留普通加权调度。
- **工具返回关系**。把运行时提供的工具调用编号与内核观察到的进程来源和阻塞事件结合起来，形成能够确认并及时删除的关系；相关进程提前使用的时间完整计入所属会话。
- **可重复的实验依据**。把 50 个公开智能体任务整理为可组合调度测试，通过完整系统比较、使用同一实现的消融实验、全部任务覆盖和异常恢复测试，分别验证会话调度、工具返回关系以及实现成本。

2 背景

2.1 共享主机上的代码智能体执行

代码智能体运行时会把每次工具调用转换为一个或多个操作系统进程。命令解释器可能启动 Python 入口，测试框架再创建多个进程并调用编译器。沙箱提供隔离，却不会替代主机处理器调度器。在混合运行的机器上，多个会话中的可运行后代最终进入同一组主机运行队列。

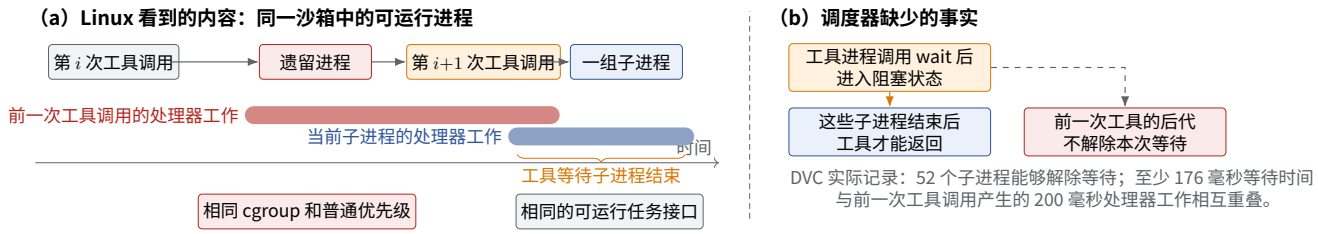


图 1: 调度器缺少的信息。Linux 能看到可运行进程和控制组，却不知道哪些后代进程结束后会解除当前阻塞等待。FlowCausal 根据内核事件重建这一关系。

运行时和内核提供互补的视角。运行时能够在会话和工具调用开始时分配编号，却无法预先知道完整进程树；Linux 能观察创建、执行、睡眠、唤醒和退出，却不知道这些事件属于哪次工具调用。面向智能体负载的主机调度器因而需要把运行时提供的编号与内核观察到的进程状态结合起来。

容器和微虚拟机保留了这一主机调度问题，同时使它更容易复现。Firecracker 提供隔离边界 [1]；AgentCgroup 将资源核算与工具调用对应 [44]；Crab 根据智能体执行轮次安排检查点保存和恢复 [37]。这些工作说明系统软件可以取得会话和工具调用编号。

2.2 Linux 的任务与 cgroup 调度

Linux 公平调度按照实体权重和虚拟运行时间分配处理器服务。cgroup 调度能够汇总多个进程并给整个组设置权重，处理器亲和性则限定进程可以在哪些处理器上运行。`sched_ext` 进一步允许 BPF 策略在不修改核心调度器的情况下选择队列、时间片和处理器 [20]。

这些机制能够使用可运行状态、已消耗的处理器时间、权重、处理器范围和控制组归属。它们说明一个实体已经获得多少服务以及可以在哪里运行，却不会给复用沙箱中的某个进程标明所属工具调用。同一个控制组可以同时包含当前工具调用、先前工具调用留下的工作进程以及长期辅助服务。

2.3 工作流与应用接口

集群工作流调度器在运行前接收作业、依赖图或资源请求 [22, 34, 38]。应用信息参与内核策略的研究也表明，单个进程之上的信息能够改进同步处理 [26]。但智能体工具的进程树在调度后才逐步形成，部分进程会在调用结束后继续存活，而且父进程可能只等待其中一部分。

因此，本文只用会话和工具调用编号标明哪些工作属于同一次调用，具体的进程关系由内核事件产生；资源权重和处理器范围仍由普通系统管理接口控制。

3 动机

3.1 调度器缺少的具体信息

仅靠父子关系、等待函数或控制组归属，都不能确定工具返回所需的进程。这里存在三种必须排除的歧义。

创建子进程不等于等待子进程。一个进程可以拥有许多仍存活的子进程，却不等待其中任何一个。若把全部后代提前执行，异步服务和先前工作也会得到同样处理。

进入等待函数不一定真的阻塞。程序会使用 `WNOHANG` 查询状态，回收已经退出的子进程，或者进行重试。只观察等待函数入口会产生错误关系。

一个控制组可以包含不同工具调用产生的进程。复用沙箱有利于保持缓存和辅助服务，但也会保留不再决定当前工具调用何时返回的进程。控制组成员关系无法区分这些进程。

这三种歧义共同限定了工具返回关系：它属于某次工具调用，只在父进程确实阻塞后成立，指出哪些存活子进程可以解除等待，并在条件变化后立即失效。

3.2 公开代码库任务中的证据

我们在查看任何 FlowCausal 结果之前检索基线调度记录。对每次工具调用，分析程序寻找三个条件：实际阻塞等待、能够解除该等待的仍存活子进程，以及先前工具调用产生且同时消耗处理器时间的后代。DVC 任务 `iterative__dvc-1494` 的第 172 次工具调用包含全部条件。当前等待的实测下界为 176 毫秒，先前工具调用产生的进程至少重叠运行 200 毫秒，能够解除当前等待的子进程共有 52 个。这些数值是下界，因为只统计调度事件能够确认的区间。

该现象准确说明了调度机会：当能够解除等待的子进程仍在争用处理器时，让其中一个进程更早运行，可以使工具调用及其会话更早完成；当子进程已经接近结束时，可调整的空间自然消失。这为实验提供了对照条件，而不是假定每次创建子进程都应带来收益。

3.3 问题定义与设计要求

本文研究的问题是：“主机调度器能否重建决定代码智能体工具调用何时返回的存活进程关系，利用它缩短会话完成时间，同时准确核算并发会话消耗的处理器时间？”这一问题把宽泛的混合运行场景收窄到工具执行期间可以获得的信息和可以采取的调度动作。

问题分析给出五项要求。**依据行为而非名称：**进程分类应使用创建和等待事件，不能依赖命令字符串。**确认实际阻塞：**只有等待进程真的睡眠后，关系才能影响调度。**保持会话间公平：**普通任务必须按会话调度，提前执行消耗的时间必须计入所属会话。**限制过期状态和调度调整：**进程退出、进程号复用、工具切换或结束消息遗漏都不能留下长期优先级。**运行期间只使用 C/eBPF：**用户态只报告低频事实，持续变化的关系和在线调度决定由内核事件驱动。

4 HarnessReplayBench

HarnessReplayBench 用于在调度器变化时保持工具执行不变。它是在公认的公开软件工程智能体任务之上新建的系统基准测试，并不替代补丁正确率排行榜。它的作用是把主机侧进程执行转化为可组合、可重复的测试输入。本文把一个来源任务及其命令保存为一份“重放记录”；把选择一份或多份记录、指定会话和控制组、

表 1: HarnessReplayBench 1.0 的任务库和测试组合。

项目	数量	在实验中的作用
来源任务	50	公开代码库任务和工具记录
代码库	42	覆盖不同构建与测试环境
工具命令	2,940	真实检索、构建、测试与服务命令
可运行任务	50	能够稳定结束并核对返回状态
单任务组合	50	独立重放和特征分析
多任务组合	34	受控竞争和规模变化

启动时间以及可用处理器的清单称为“测试组合”。这样，任务本身与多个任务共享机器的方式彼此独立。

4.1 来源任务与覆盖范围

任务库包含 50 个来源任务、42 个代码库和 2,940 条工具命令。其中 46 个任务来自 SWE-rebench，3 个来自 SWE-bench Lite，1 个来自公开的 SWE-agent 执行记录。原始执行包括 12 个 OpenHands 任务、8 个 SWE-agent 任务和 30 个 mini-SWE-agent 任务。这些来源提供公开代码库版本、可执行容器和真实工具序列，而不是人工编写的处理器循环。

50 个任务都能在版本固定的软件环境中结束，返回状态符合预期,内核事件丢失为零。对每个任务,我们还在命令和可用处理器相同的条件下, 记录其在 `scx_simple` 下的进程创建、退出和处理器执行情况。三份导入记录只修复运行环境，不改变计算内容：Dask 使用适合标准输入程序的多进程启动方式，Sanic 保存工作进程输出，DVC 从同一任务和镜像重新建立工作区。三者现在都能正常结束，返回状态正确且没有丢失内核事件。

这些命令覆盖代码库查看、一般命令、编辑、测试、软件包管理和构建，并可能继续启动编译器、工作进程池或本地服务。经过验证的单任务执行从 39 个进程到 115,080 个进程不等，耗时从不足一秒到超过一千秒。因此, 测试组合可以根据进程结构和执行长度选择任务, 而不需要修改任务本身。

4.2 重放记录与执行条件

每份重放记录包括容器镜像、代码库状态、工作目录、环境、命令、启动时间、预期返回类型、超时时间和清理方法。命令仍在原来的软件环境中执行，编译器和测试框架不会被休眠或空循环代替。运行程序保存四类记录：运行时报告的工具调用起止信息、C/eBPF 记录的进程创建和调度事件、命令返回状态、资源使用情况。只有每条命令的返回状态属于重放记录允许的类型、清理成功且内核事件丢失数为零时，该次运行结果才会进入统计。

发布包提供 50 个单任务组合和 34 个多任务组合。清单明确列出任务、会话和控制组，指定同时启动或错峰启动，并固定允许使用的处理器。任务库与测试组合相互独立，后续研究可以增加新组合而无须改写任务记录。

导入程序先把公开任务绑定到代码库版本和镜像, 再整理每条命令的工作目录、环境、执行方式和预期返回类型。随后在 `scx_simple` 下单独执行任务, 确认命令序列能够结束并记录进程与处理器事件。测试组合再把不变的任务编号与启动时间和处理器设置结合。单任务记录产生的离线标注只用于实验结束后的分析；FlowCausal 运行时只能使用实际部署中也能取得的运行时消息和内核事件。

4.3 避免依据候选方案选择任务

主要实验使用两个互补的四会话组合。“混合工作流组合”同时运行 Dask、DVC、Numba 和 SQLFluff，用来检查按会话调度在并发启动时的作用。“工具返回组合”完全依据 `scx_simple` 的记录选择，选择条件正是第 3节给出的实际阻塞、能够解除等待的存活子进程, 以及先前工具调用产生的进程同时使用处理器。选择过程不读取任何 FlowCausal 耗时结果。

另外两个组合用来判断机制是否因所述原因生效。“构建等待组合”包含仍在继续运行的构建进程；“短子进程组合”中，能够解除等待的子进程只运行 7.75 毫秒且只有一个，理论上没有明显调度空间。若一个机制在这里仍得到相同收益，更可能是对整个组合施加了优先级差异，而不是准确利用工具返回关系。

4.4 评价指标

本文使用两项用户可直接感知的完成时间指标。**最后一个工作流完成时间**也常称为总完成时间，从测试组合启动开始，直到最后一个会话结束。**平均工作流完成时间**是组合中所有会话耗时的平均值，能够反映部分会话是否更早完成。两项指标都是越低越好。本文还分别报告工具返回关系中的进程识别率、事件丢失、处理器开销、内存使用和状态恢复测试，不把完成时间、关系识别和实现成本合并为一个指标。

4.5 重放一致性与可复现性

调度器基准不能只比较命令名称是否相同。HarnessReplayBench 从三个层面核对条件。执行一致性要求可执行命令、工作目录、解释器环境、代码库状态和容器镜像一致；进程一致性核对进程创建与执行链、最大存活进程数量和命令结果；组合一致性固定会话归属、启动方式、允许的处理器和清理方法。这样，完成时间变化才能归因于处理器调度, 而不是程序走了不同路径。

不同公开来源记录环境的详细程度不同。重新采集的任务恢复记录中的解释器、环境变量和命令解释器设置；导入记录缺少这些字段时，使用配套固定版本镜像中的测试环境。该规则写入重放记录并对所有调度器保持相同。来源记录中允许失败的探索命令可以返回非零值，但任何意外失败都会使该次运行结果失效，调度器不能依靠少执行命令或提前失败表现得更快。

全部容器到达统一的就绪点后才开始计时。成功结果保留事件、命令结果、调度器计数和摘要；失败结果保留完整工作区和输出。版本 1.0 的重放记录保持不变, 测试组合另行指定任务、启动时间和处理器。增加新组合不会改变已有结果。

5 系统总览与设计依据

图 2 展示信息如何从沙箱运行时进入处理器选择。系统采用两项划分：在识别方面，运行时提供稳定的会话和工具调用编号，内核提供不断变化的进程与等待事件；在调度方面，会话之间的处理器时间分配与会话内部可运行进程的短期次序分别处理。这样可以让工具调用更早返回，又不会把一次短暂关系变成永久优先级。

运行时登记。每个会话位于独立 cgroup，同一工作流的进程由此归为一组。`flowd_v2` 核对 cgroup 标识、根进程号及会话和工具调用的起止消息, 再更新 BPF 映射。cgroup 确定进程所属会话，工具调用编号与实际阻塞事件进一步确定当前等待关系。

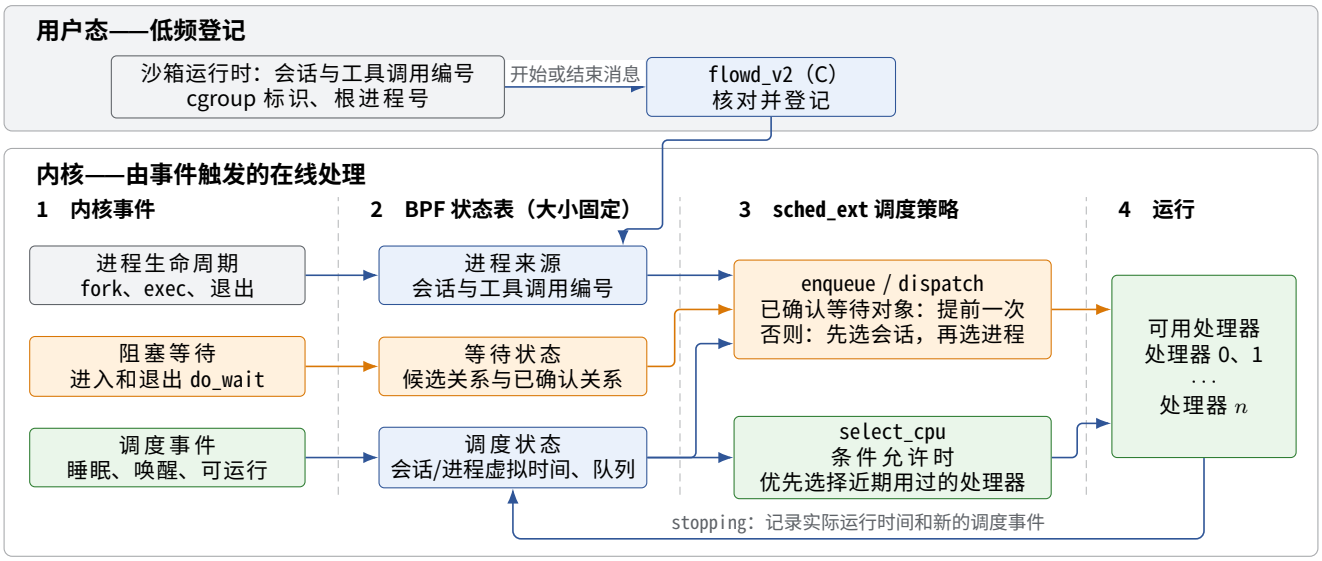


图 2: FlowCausal 的系统结构和反馈过程。C 程序登记会话、工具调用和 cgroup 标识。内核事件更新大小固定的 BPF 状态表；`sched_ext` 回调根据这些状态选择进程和可用处理器；随后记录实际运行时间，供下一次调度使用。

内核观测。eBPF 让登记编号随 `fork` 与 `exec` 传递。进入 `do_wait` 时只建立候选关系，随后观察到进程睡眠才确认阻塞；生命周期事件负责删除数量受限的状态。

处理器调度。普通路径按照加权虚拟运行时间选择会话，再选择会话内进程。已经确认的工具返回关系可以让一个进程提前获得短时间片，其实际运行时间仍计入所属会话。与当前等待竞争的先前工作也可以短暂延后。这两种动作都不改变会话权重。

处理器选择。进程运行后 250 毫秒内，调度器优先尝试它近期使用且仍可用的处理器；负载、亲和性和可用状态仍具有更高优先级，因此不会形成固定绑核。

6 核心机制

6.1 记录不同工具调用产生的进程

设计理由。 运行时启动工具时知道根进程，但后来真正影响返回的进程可能是经过多次 `fork/exec` 产生的后代。周期性扫描整棵进程树的工作量会随历史进程数量增长，还可能错过寿命很短的进程。因此，FlowCausal 在创建关系的事件发生时增量维护来源信息。

具体做法。 C 程序登记会话编号、递增的工具调用序号、根进程号和 cgroup 标识。创建新进程时，子进程继承父进程的会话编号和工具调用序号；`exec` 改变程序，但不改变来源。进程退出后删除存活记录。映射键同时保存进程号和启动时间，防止进程号复用把新进程错误连接到旧会话。工具调用结束后，本次序号关闭；仍存活的后代保留原来源，因而能与下一次工具调用产生的进程区分。

这里记录的只是进程来自哪次工具调用，不代表它应当更早运行。只有通过下文的阻塞等待确认后，后代才会进入工具返回关系。这一区分避免把“所有后代都重要”变成长期提权策略。

6.2 确认真正的阻塞等待

设计理由。 内核等待路径能够直接说明工具正在等待进程结束，但只看到函数入口仍不够。`WNOHANG`、已经结束的目标、信号和重试都可能使等待立即返回。因此，FlowCausal 将函数事件与调度器事件结合。

候选与确认。 `do_wait` 的 fentry 程序记录等待进程和选择条件，带 `WNOHANG` 的调用直接忽略。如果调度器随后发现该任务在候选项仍有效时离开处理器并进入睡眠，关系才被确认；fexit 程序删除立即返回的候选项。这样就不会把一次没有改变可运行状态的函数调用误认为进程依赖。本文把候选等待产生后、直到等待函数返回或等待进程被唤醒的这段过程称为“一轮等待”；后文只用它限制同一次等待中可以重复执行的动作。

选择相关进程。 若等待指定了明确的进程号，系统只在目标仍存活且属于同一会话和同一工具调用序号时使用它。对于等待任意子进程的 `wait(-1)`，系统增量查询近期仍存活的直接子进程，不扫描完整后代树。若选中的子进程本身也被确认处于阻塞等待，可以继续扩展一层。当前实现最多选择四个进程、最多跟踪两层等待。例如，工具进程等待构建进程，构建进程又等待编译器，这就是两层等待。这里的目标是识别少量能够解除当前等待的工作，而不是在 BPF 中建立没有上限的依赖图。

删除关系。 等待进程重新可运行、等待函数返回、目标退出、工具调用序号改变或会话结束时，关系都会被删除。序列号会拒绝先前一轮等待中迟到的更新。及时删除与建立关系同样重要，否则一次短暂观测会变成长期优先级。

图 3 概括了两个判断：只有等待进程确实进入睡眠，候选项才会变成正式关系；只有相关进程和工具调用仍然有效，关系才会保留。

6.3 按会话调度与有限提前运行

为何普通任务按会话调度。 如果全部可运行进程在全局竞争，拥有大型测试进程池的会话会因为暴露更多任务而获得更多处理器时间；如果把工具返回关系中的所有子进程都排在其他会话之前，一次涉及大量子进程的等待又可能独占主机。FlowCausal 因此使用加权虚拟

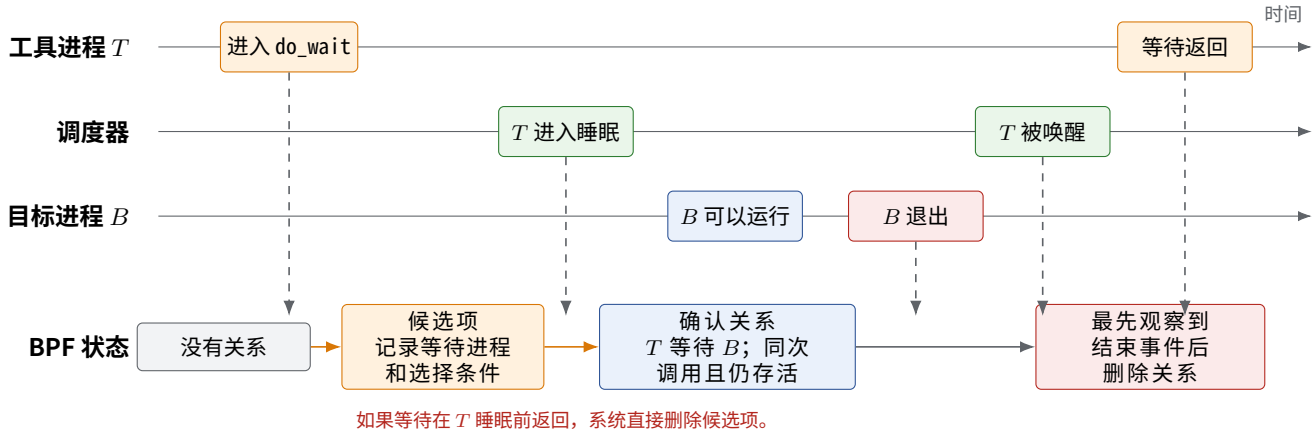


图 3: 根据内核事件的先后顺序确认工具返回关系。上方三行是内核事件，下方一行是 BPF 状态。进入等待时只产生候选项，随后观察到进程睡眠才确认关系；最先出现的结束事件会删除关系。若等待在进程睡眠前返回，则不会建立关系。

运行时间，从一个活动会话中选择普通任务。会话权重仍由系统管理者决定；仅仅创建更多进程不能增加该权重。

为何允许一个进程在普通顺序之前运行。 如果一定等到所属会话再次被选中，解除工具等待所需的进程仍可能排在其他会话之后。因此，工具返回关系中的一个进程可以进入全局“提前执行队列”，获得一个 2 毫秒时间片。这里的“提前”只表示在普通会话选择之前检查该队列。这不是不计成本的高优先级：进程停止运行时，调度器测量它实际使用的时间并计入所属会话的虚拟时间，下一次普通会话选择会反映这段提前使用的处理器时间。会话内部默认仍按每个进程的虚拟运行时间排序；先前工具调用产生且与本次返回无关的进程可以短暂延后，但不会被移除，也不会改变控制组权重。

限制与恢复。 每个会话都有提前执行的时间上限。默认设置只处理一个直接等待对象，同一进程在一轮等待中最多获得一个提前时间片。事件测试还覆盖两层嵌套等待和最多四个进程。只有父进程已经确认处于等待状态且下一层进程变为可运行时，下一层进程才会被选中。全部公开任务性能实验统一只处理直接等待对象，避免不同组合使用不同等待层数。被延后的任务必须在规定时间后重新得到普通调度。来源或等待状态缺失、矛盾或过期时，进程按照普通虚拟运行时间排序。因此，观测不完整只会失去优化机会，不会破坏程序正确性。

优先复用近期处理器。 进程刚运行过的一小段时间内，若上次使用的处理器空闲且符合亲和性设置，FlowCausal 会优先选择它，以尽量保留指令和数据缓存。该选择只是偏好：负载、亲和性和处理器可用状态具有更高优先级，因而不会产生固定绑核导致的计算能力损失。

图 4 给出了完整顺序。调度器先检查有明确限制的提前执行队列；若队列中没有可运行进程，再依次选择会话和会话内进程。两条路径使用同一套处理器时间核算。

表 2: 内核事实、调度动作及其设计理由。

观察到的事实	调度动作	理由
创建子进程	继承会话编号与工具调用序号	不扫描即可保存来源
进入等待后睡眠	确认进程关系	排除非阻塞查询
关系中的子进程可运行	一个提前时间片	尽快完成解除等待的工作
先前工具调用的进程可运行	暂时延后	减少与当前等待的竞争
近期使用的处理器可用	优先复用	不绑核地保持局部性
唤醒、退出或切换工具	删除状态	防止过期优先级

7 系统实现

7.1 调度器与状态组织

为何分别保存会话状态和任务状态。 FlowCausal 由 `sched_ext` 调度器、BPF 事件程序和 C 登记程序组成。调度器把所有活动会话 cgroup 放在一棵按加权虚拟运行时间排序的树中。每个 cgroup 状态表示一个会话；每个任务状态保存虚拟运行时间、会话编号、工具调用序号、近期处理器和等待轮次信息。每个会话的调度队列保存普通任务，后备队列保存未登记任务；全局提前执行队列只保存由已确认工具返回关系选出的进程。后者实际消耗的运行时间由停止运行回调记录，并计入对应会话。单独的会话状态保存已经确认的等待，以及普通会话选择使用的设置权重。

为何只在等权条件下省略重复核算。 普通会话选择同时使用虚拟运行时间和周期处理器时间核算。会话权重不同或会在线改变时，后者不可缺少；但所有已登记会话使用同一个固定权重时，两者表达的是同一个等权约束。C 程序因此维护当前基础权重是否全部相同。只有等权且固定权重时，eBPF 才跳过重复的周期核算；一旦权重不同或在运行期间改变，就继续使用完整核算。该优化不改变队列次序和资源分配规则。

为何所有状态都有明确上限。 表 3 列出实验采用的固定范围。它们限制两类成本：映射容量限制常驻状态，即

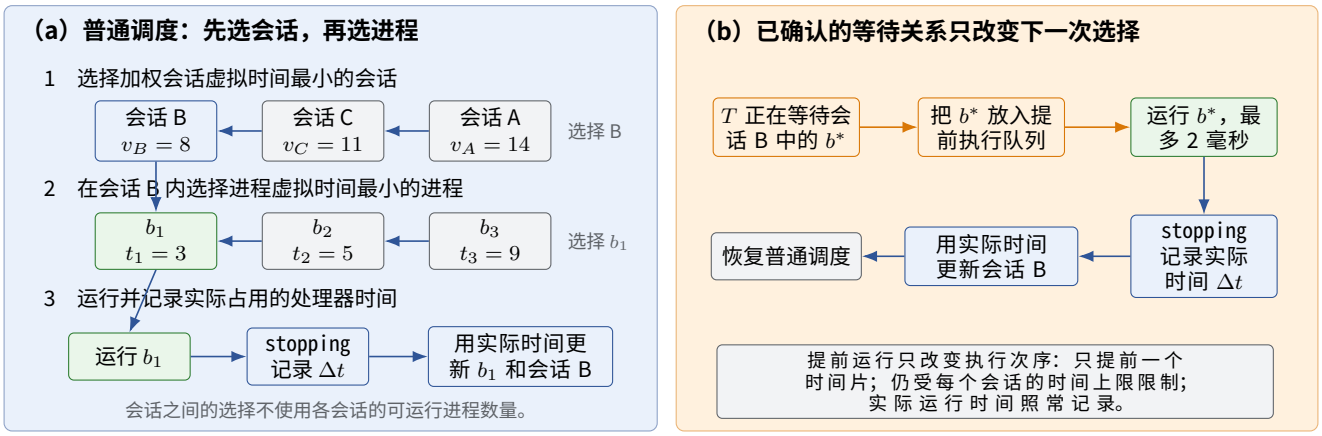


图 4: 进程选择和处理器时间记录示例。普通调度按照虚拟时间选择会话 B，再选择 b_1。已经确认的等待关系可以改为让 b^* 提前运行一次，最长 2 毫秒。两种方式都按实际运行时间更新所属会话，然后继续普通调度。

表 3: 实验采用的实现范围。

状态或动作	上限	作用
会话记录 / 恢复位置	4,096 / 64	限制状态和异常恢复遍历
存活任务记录	131,072	限制进程来源状态
工具调用记录	16,384	区分重叠的工具调用
近期直接子进程	每任务 8 个	无须扫描即可处理 `wait(-1)`
关系中的进程	1 个 / 4 个、2 层	性能实验 / 事件测试；限制事件处理
提前执行时间上限	每 100 毫秒 10 毫秒	限制每个会话提前使用的处理器时间
提前时间片 / 暂时延后	2 毫秒 / 20 毫秒	限制单次动作
会话登记超时 / 处理器复用	2 秒 / 250 毫秒	删除失效的登记和局部性状态

使运行时没有报告结束也不会无限增长；更小的工具返回关系范围限制一次内核事件的处理量和能够提前运行的进程数。64 个活动会话位置只在虚拟时间树状态不一致、需要恢复某个会话队列时检查。普通会话选择直接使用虚拟时间树，不遍历该数组。

7.2 事件处理与调度路径

为何在事件发生处更新状态。 事件程序连接进程创建、执行、退出跟踪点，调度状态变化事件，以及 `do_wait` 周围的 fentry/fexit。BPF 映射把进程号和启动时间组成的身份连接到会话编号与工具调用序号。固定大小的近期子进程记录用于处理 `wait(-1)`。每个会话的计数器记录候选等待、确认睡眠、建立关系、提前时间片、暂时延后、近期处理器选择、后备调度和过期状态清理。基准测试只在一轮结束后读取这些计数，不会把它们反馈给调度决策。

BPF 程序把事实收集与高频调度分开。进程创建、程序替换、等待、切换和退出事件只在事实变化时更新任务记录。`runnable` 回调把新近可运行的子进程与当前睡眠的等待进程对应；`enqueue` 在所属会话队列、提前执行队列和后备队列之间作出选择；`select_cpu` 尝试复用近期处理器；`dispatch` 最多先取一个提前执行队列中的进程，随后继续按权重选择会话。最后，`stopping` 记录实际运行时间，退回没有使用的提前执行时间，并删除单时间片标记。调度回调不会遍历进程树，也不会遍历会话中的全部任务。

为何正常路径不进行详细测量。 实现还把论文机制与详细测量分开。正常版本只更新下一次调度所需的状态；单独的诊断选项才会记录每类进程的运行和排队次数。论文中的主要性能结果使用正常版本。开销实验使用两个构建方式相同的版本：一个关闭工具返回关系及其调度动作，另一个启用全部机制。因此，报告的差值表示维护工具返回关系的新增成本，不包含实验结束后的离线统计程序。

为何调度状态保留在 BPF 中。 调度器与观测程序共享同一个有界任务记录，不在用户态重复建立运行中的状态模型。这样做有两点实际意义。第一，进程入队时能够直接读取刚由内核事件产生的关系，不受轮询周期和事件复制往返影响。第二，BPF 校验器能够依据活动会话扫描上限和工具返回关系上限确定循环次数及内存范围。C 程序负责更适合普通错误处理的工作，例如核对新会话并打开控制组；高频调度判断仍留在 BPF 中。

7.3 C 程序登记与异常恢复

为何保留一个小型 C 程序。 `flowd_v2` 使用 libbpf，以本地 Unix 套接字接收登记消息，核对控制组和进程身份后更新映射。它不判断进程类别，不遍历 `/proc`，不轮询调度状态，也不选择处理器。若该程序退出，会话登记的定期更新时间随之停止；两秒超时后，相关任务自动使用后备调度路径。Python 只用于实验程序启动容器和离线汇总，不属于所部署的调度器或运行期间的用户态程序。

为何把事件测试与性能实验分开。 实现包含三层测试。事件级测试覆盖 fork 继承、exec 保持、非阻塞等待排除和状态清理；状态恢复测试主动结束 C 程序、遗漏一次

工具调用结束消息以及强制目标退出；端到端重放要求命令结果符合预期且内核事件丢失为零。这种结构也体现 kSTEP 关于调度器开发需要确定性事件测试、不能只看吞吐数字的经验 [5]。

7.4 CPU 服务核算与重新分派

对于同进程 futex 唤醒，短计算线程获得的提前运行额度受实际睡眠时间限制，避免每次唤醒重复获得完整的 20 毫秒额度。临时会话权重的 CPU 服务核算从任务开始执行时刻与权重生效时刻的较晚者计算，避免将权重生效前的服务按新权重计入。最后一个可运行任务主动让出 CPU 后重新请求分派，仍经过会话份额和硬配额检查。动态并行运行限制只约束同时在 CPU 上运行的任务数量，不修改应用创建的进程数量。

8 实验评价

实验将引言中的三项贡献逐一对应到证据。**研究问题一（会话调度）：**把智能体会话作为处理器时间的长期分配单位，能否比 Linux 和按进程调度的 `sched_ext` 调度器更早完成工作流？**研究问题二（工具返回关系）：**依据工具返回关系安排进程次序，能否在会话调度之上继续产生收益，什么条件下存在这种收益？**研究问题三（适用范围与实现成本）：**同一套固定策略能否适用于不同处理器数量、会话数量和全部任务，并把处理器开销、状态数量和异常情况下的退回处理控制在实际可用范围内？以下每项结果只回答对应问题，消融实验不被当作外部调度器对照。

8.1 实验方法

实验机器。 实验主机配备两路 Intel Xeon Platinum 8375C，共 128 个逻辑处理器，可用内存 256 GiB，运行 Linux 6.17，处理器频率策略设为 performance。除规模实验外，全部工作流和调度器都限制在 0–7 号处理器上，即同一 NUMA 节点上的八个物理核；这些核的超线程兄弟不在所选集合中。同一组合的全部会话共享这八个处理器，不对各会话分别绑核。

对比调度器。 本文选择三种主要对照：（1）Linux 默认公平调度；（2）上游 `scx_simple`，本实验采用其默认的全局加权虚拟运行时间模式，它是 `sched_ext` 的基础示例；（3）Linux 下每个工作流处于独立同权 cgroup。Linux 代表生产环境默认设置；`scx_simple` 用来在相同调度框架中评估 FlowCausal 加入的运行信息是否有效；同权控制组用于区分“建立控制组”和 FlowCausal 进程排序的作用。“仅会话调度版本”的 FlowCausal 使用相同代码和按会话加权调度，但关闭工具返回关系、提前执行、暂时延后以及近期处理器复用。该版本只用于衡量这些机制带来的额外作用，不把它当成外部基线。

我们也尝试了上游 `scx_flatcg`。限制处理器范围后，两批各五轮实验都出现系统任务超过 30 秒没有运行，内核随后卸载该调度器。后续工作实际由 Linux 完成，因而这些结果不能组成 `scx_flatcg` 的性能均值。Linux 同权控制组提供稳定的控制组对照，仅会话调度版本则单独衡量 FlowCausal 中对应机制的作用。

运行方式。 不同方法使用完全相同的清单、容器镜像、控制组、可用处理器、启动时间和超时设置。两组四会话完整系统比较以及工具返回关系机制实验中，每种方法运行五次；较大的规模测试运行三次。每轮开始前清理页缓存；在同一系列中轮换方法顺序；失败结果保留现场，成功结果提取数据后删除可重建工作目录。表中报告算术平均值与样本标准差。所有进入论文的运行均符合任务预期返回类型、完成清理并且没有丢失内核事件。

表 4: 混合工作流组合（平均值 ± 样本标准差，五轮）。

调度器	最后完成（秒）	平均工作流（秒）
Linux	92.411 ± 0.613	63.269 ± 0.682
Linux+cgroup	92.245 ± 1.181	63.797 ± 0.993
`scx_simple`	101.872 ± 0.740	88.990 ± 0.633
FlowCausal	$\mathbf{90.857} \pm 1.011$	$\mathbf{60.621} \pm 0.236$

每个会话位于独立 cgroup 且权重均为 100，全部会话共享同一个可用处理器集合。`scx_simple` 不使用控制组身份，但执行相同任务并使用相同处理器集合。所有方法都运行相同的 C/eBPF 记录程序。只有 FlowCausal 还运行 C 登记程序，其处理器时间计入开销结果。`scx_simple` 是使用相同框架的主要对照，Linux 默认调度是部署参考。

8.2 研究问题一（a）：完整系统与调度基线

主要结论：在“混合工作流组合”中，FlowCausal 比所有外部对照更早完成最后一个工作流，平均工作流完成时间也最短。图 5 展示四会话组合中每种方法的五轮交错实验。相对 `scx_simple`，完整 FlowCausal 将最后一个工作流完成时间从 101.872 秒降至 90.857 秒（10.80%），平均工作流完成时间从 88.990 秒降至 60.621 秒（31.88%）。两项逐轮配对变化的 95% 置信区间分别为 [8.95%, 12.66%] 和 [31.04%, 32.71%]。

Linux 默认调度的最后一个工作流和平均工作流完成时间分别为 92.411 秒和 63.269 秒；FlowCausal 分别缩短 1.68% 和 4.17%，逐轮配对区间为 [0.56%, 2.81%] 和 [2.51%, 5.84%]。Linux 使用同权控制组时的两项结果为 92.245 秒和 63.797 秒，与默认调度接近。FlowCausal 相对它分别缩短 1.48% 和 4.96%。由此可见，FlowCausal 的作用不只是建立控制组，还包括在会话之间分配处理器时间，以及在会话内部安排进程次序。

第二组公开任务。 “工具返回组合”保留 Dask、DVC、同时启动和 0–7 号处理器，用第二个 Numba 任务替换 SQLFluff。FlowCausal 相对 Linux 将最后一个工作流和平均工作流完成时间分别缩短 2.58%（区间为 [1.28%, 3.87%]）和 5.23%（区间为 [3.04%, 7.42%]）；相对 `scx_simple` 的平均工作流完成时间缩短 27.86%。相对 `scx_simple` 的最后完成时间差异为 0.54%，其区间跨过零。因此，两种任务组成相对 Linux 的两项指标都得到改善，而“混合工作流组合”给出了相同调度框架下更明确的最后完成时间收益。

8.3 研究问题一（b）：会话调度的作用

主要结论：把智能体会话作为处理器时间的分配单位，可以避免进程数量多的工作流仅凭任务数量占用更多处理器。一组独立的处理器时间实验解释了为什么需要按会话分组。任务组 A 保持 2 个可运行进程，任务组 B 保持 16 个可运行进程。两组位于同一控制组时，Linux 为 A 分配 11.14% 的处理器时间；把两组放入两个同权控制组后，A 的占比提高到 21.83%。在 `scx_simple` 下，

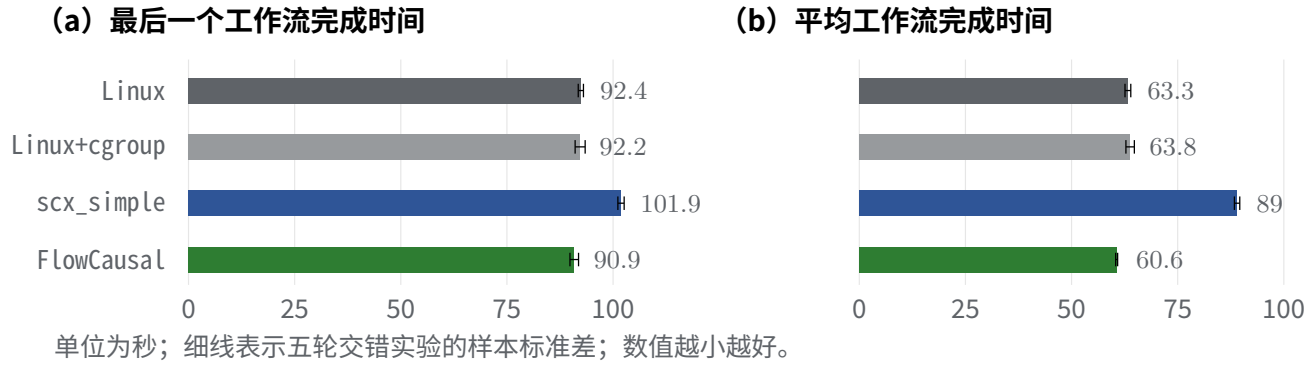


图 5: “混合工作流组合”的完整结果。标签给出五轮绝对平均时间，细线表示样本标准差，数值越低越好。

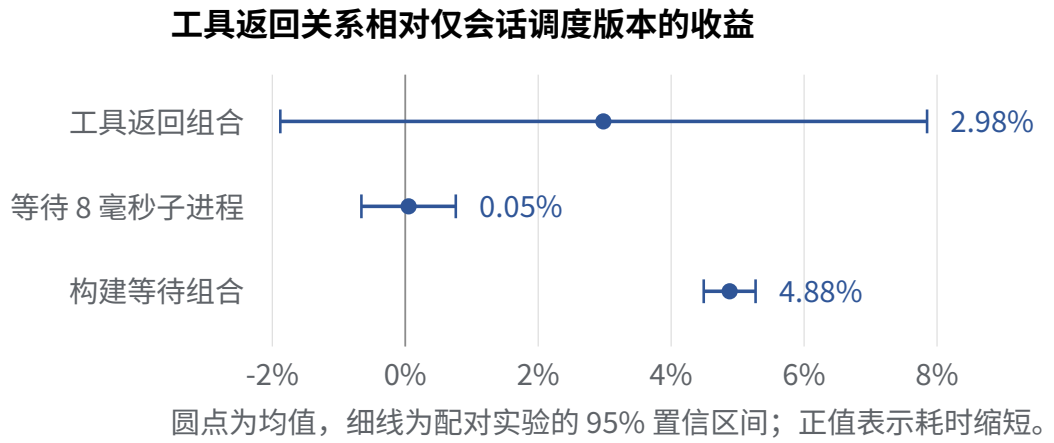


图 6: 工具返回关系相对仅会话调度版本的平均工作流完成时间收益，每种方法五轮。“构建等待组合”稳定改善，被等待子进程只运行约 8 毫秒的组合接近不变。

两种设置均约为 11.13%，与按进程数量分配的结果一致。因此，全局进程队列会让进程多的工作流仅凭进程数量取得更多运行机会；先选择会话可以消除这一影响。主要组合中，`scx_simple` 与 FlowCausal 的平均工作流完成时间相差 28.72 秒，而 Linux 已经取得按控制组分配带来的大部分作用。下一组实验再单独衡量工具返回关系在会话调度之上增加的作用。

8.4 研究问题二：工具返回关系的额外收益

主要结论：当解除工具等待所需的进程仍在竞争处理器时，依据工具返回关系调整进程次序能够加快完成；当这段工作极短时，结果接近不变。图 6 使用同一调度器实现，对比完整 FlowCausal 与仅会话调度版本。在“工具返回组合”中，加入工具返回关系后平均工作流完成时间缩短 2.98%，95% 置信区间为 $[-1.88\%, 7.85\%]$。在“构建等待组合”中，工作流等待处理器密集的构建子进程结束后还要继续运行，工具返回关系使平均工作流完成时间缩短 4.88%，区间为 $[4.49\%, 5.27\%]$。这些任务都在查看候选调度器耗时之前，依据 `scx_simple` 的内核事件记录选出。

短子进程组合提供了有区分力的对照。选中的子进程只运行 7.75 毫秒，关系中只有一个任务。完整 FlowCausal 相对仅会话调度版本的平均工作流完成时间变化为 0.05%，区间为 $[-0.66\%, 0.76\%]$，接近预期的无明显变化。由此可见，该机制并非简单地提高当前工具调用的整体优先级；只有能够解除等待的子进程还需要足够多的处理器时间时，重新排序才会产生收益。

调度器计数进一步说明动作确实来自这一路径。在混合组合中，每轮约记录 10,983 个带有会话编号和工具调用序号的进程事件，以及 1,236 次确认睡眠等待，却只实际提供 31.7 个提前时间片，并暂时延后先前工具调用产生的进程 2.7 次。观测数量与动作数量之间的差距是有意设计的：来源和睡眠确认先缩小候选范围，每轮和每会话限制再使最终调度调整保持稀疏。

8.5 研究问题三（a）：处理器数量与会话数量

主要结论：FlowCausal 在标准和较多处理器设置下都能缩短相对 Linux 的平均工作流完成时间；八个会话同时运行时收益仍然存在。图 7 使用“工具返回组合”。四个会话使用 4、8 和 16 个处理器时，完整 FlowCausal 相对 Linux 将平均工作流完成时间分别缩短 3.22%、5.23% 和 5.66%。三轮 4 处理器实验的区间跨过零，8 和 16 处理器实验的区间保持为正。所有点都直接以 Linux 为参照，避免把 `sched_ext` 框架本身与 Linux 的差异误当成 FlowCausal 算法收益。

在八个处理器上把并发会话增至八个后，FlowCausal 相对 Linux 将平均工作流完成时间缩短 3.45%，区间为 $[1.95\%, 4.96\%]$。相对仅会话调度版本，完整 FlowCausal 又将最后一个工作流和平均工作流完成时间分别缩短 1.67% 和 1.61%，两项区间都保持为正。这正是 FlowCausal 面向的混合运行情况：多个进程树同时可运行，但每次工具等待只有一组受限进程决定返回。全部八个会话共享同一处理器集合，没有为任何工作流分配专用处理器。

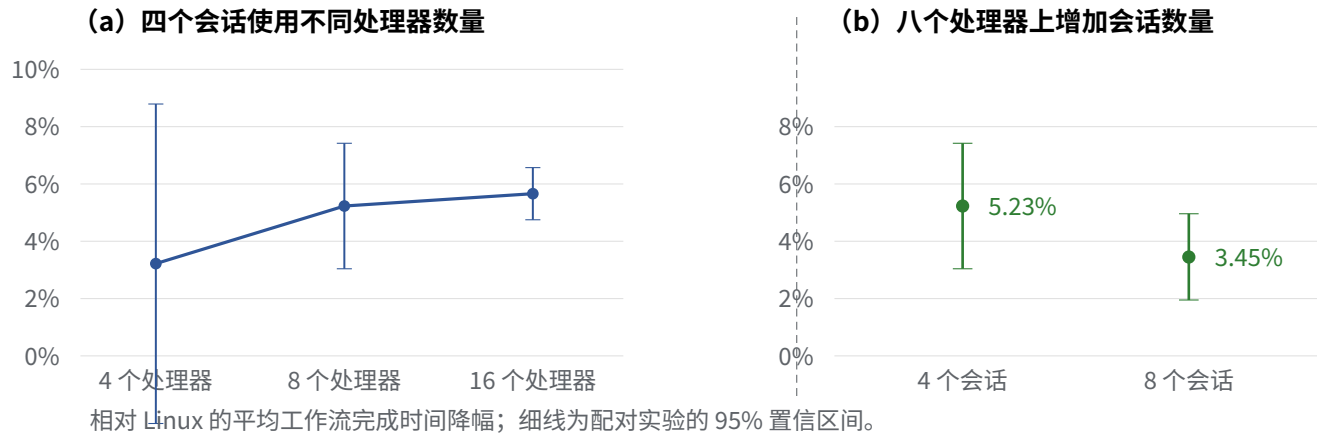


图 7: 工具返回组合的规模变化。所有数值均为相对 Linux 的平均工作流完成时间降幅，细线为逐轮配对的 95% 置信区间。处理器规模测试使用四个会话，会话规模测试使用八个处理器。

表 5: 工具返回组合中的机制证据与开销。

测量项目	结果
工具返回关系中的进程识别率	99.04%
丢失内核事件	0
新增 BPF 处理器开销	0.445%
完整 BPF 处理器开销	1.173%
BPF 单次调用平均耗时	864 纳秒
C 登记程序处理器开销	0.0292%
C 登记程序合计最大常驻内存	8.8 MiB
任务结果检查	全部通过
状态恢复测试	全部通过

8.6 研究问题三（b）：全部任务的覆盖检查

主要结论：同一套固定策略能够覆盖任务库中的全部工作流，不需要根据代码库或命令名称编写专用规则。我们把 50 个任务分为 10 个五会话组合，每组在 0–7 号处理器上同时启动，每个任务恰好出现一次。当前 FlowCausal 完成了全部 10 个组合和 2,940 条命令。汇总十组结果，最后一个工作流完成时间相对 Linux 和 `scx_simple` 分别缩短 0.20% 和 0.31%，平均工作流完成时间分别缩短 0.26% 和 1.66%。这是覆盖范围检查而不是显著性结论：策略没有使用代码库名称，主要实验和机制实验中的多轮结果仍负责证明性能收益。

8.7 研究问题三（c）：成本、关系识别率与恢复

主要结论：完整在线路径约占所测八个处理器总计算能力的 1.2%，状态缺失时会自然退回普通排序。在“工具返回组合”中，完整 BPF 路径累计消耗 7.929 秒处理器时间，占八个处理器总计算能力的 1.173%。仅会话调度版本占 0.727%，其余机制新增 0.445 个百分点。917 万次 BPF 调用的平均耗时为 864 纳秒；四个 C 登记程序占总计算能力的 0.0292%，合计最大常驻内存为 8.8 MiB。

会话等权时，跳过重复的周期核算使 `dispatch` 平均耗时降低 15.5%，BPF 总处理器时间降低 10.1%。权重不同或运行期间发生变化时仍使用完整核算；重点任务和全部任务复验确认该处理没有改变调度结果。

离线分析在实验结束后重建每次阻塞工具调用可以由哪些进程解除等待；这些信息不会交给在线调度器。在线机制识别出其中 99.04% 的进程。漏掉一个进程只会减少优化机会，不会使它失去普通虚拟运行时间调度。恢复测试覆盖非阻塞等待、目标退出、工具调用结束消息遗漏、C 程序退出以及延后进程恢复普通调度。所有性能实验的任务结果均符合预期，内核事件丢失数均为零。

9 跨命令依赖的调度与评估

9.1 旧命令进程成为当前等待对象

先前命令留下的后台进程可能被当前命令通过 `waitpid` 等待。如果调度器仍按旧后台身份安排该进程，它就可能延迟已经依赖于它的当前工具调用。为研究这种依赖变化，将跨命令处理作为单独的实验原型，与 FlowCausal 的默认实现进行对照。

原型对后台成员合计分配份额；确认某个成员成为当前工具的等待对象后，在其下一次入队时消除旧后台身份积累的组内排队惩罚。实际 CPU 服务仍计入所属会话，原有工作流份额、并行运行限制、硬配额和提前运行预算继续生效。这一处理改变会话内的执行次序，不增加该会话的总资源份额。

9.2 固定工作量消融

机制基准创建 2 个工作流，每个工作流先等待新命令子进程，再等待旧命令子进程。测试不向调度器预告后续等待对象；完成时间包含所有后台工作，计算结果通过校验值核对。计时从进程登记后的统一放行开始。主配置包含 2 秒模拟非 CPU 阶段，不包含真实 I/O 或完整应用启动成本。

在 4 个 CPU、8 个后台进程的同批消融中，每种方案运行 3 轮。原生 Linux 的平均完成时间为 4.828 ± 0.116 秒，完整原型为 4.065 ± 0.009 秒，降低 15.79%。关闭跨命令关联后为 4.925 ± 0.007 秒，关闭后台合计份额后为 4.907 ± 0.015 秒。后台进程数为 16 时降低 28.66%，为 2 时基本持平；移除模拟等待的对照降低 7.63%。结果说明，该机制的收益与依赖转换及后台竞争程度有关。

9.3 八工作流对照

在 4 个 CPU 上并发运行 8 个工作流，以交错顺序比较原生 Linux、FlowCausal 和跨命令依赖原型，每种方案运行 3 轮。四会话实验与本组实验采用各自固定的实现及运行配置；本组实现包含上述唤醒额度、CPU 服务核算与重新分派处理。表 6 报告轮间均值和样本标准差。

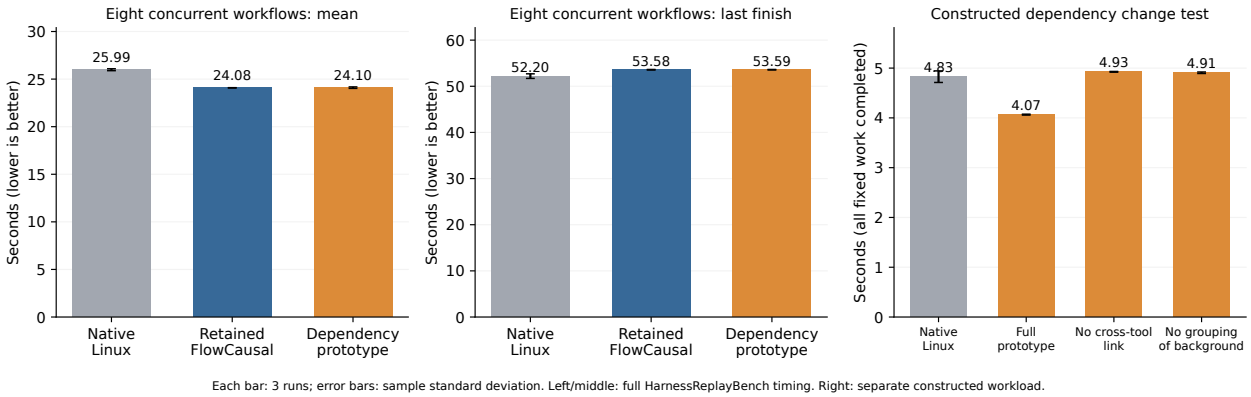


图 8: 跨命令依赖的固定工作量消融与八工作流对照。两类实验分别测量依赖转换的作用与完整工具执行的完成时间。

表 6: 八工作流完成时间，单位为秒；均值 ± 样本标准差，3 轮。

方案	平均工作流完成	全部完成
原生 Linux	25.986 ± 0.100	52.201 ± 0.497
FlowCausal	24.084 ± 0.016	53.584 ± 0.057
跨命令依赖原型	24.103 ± 0.078	53.587 ± 0.053

FlowCausal 相对原生 Linux 将平均工作流完成时间降低 7.32%，全部完成时间增加 2.65%。跨命令依赖原型的平均完成时间与默认实现接近；后台合计份额与依赖转换修正的关键计数均为零。因此，这组真实工具回放未显示该原型的额外收益，回放中的总体改善不能归因于跨命令处理。

实验仅统计实际调度器与计划一致的样本；原生 Linux 对照禁止配置调度器二进制或工具控制器，并在运行时检查调度器状态。机制实验共包含 66 次运行，八工作流对照包含 9 次运行，各组分别统计。标签与实际调度器不一致的试验不进入性能结果。

9.4 作用条件

固定工作量实验直接构造从后台进程到当前等待对象的依赖转换，能够触发对应调度动作；真实工具回放中未触发这些关键计数。两类结果表明，该原型仅在特定依赖和竞争条件下产生收益。I/O 优待筛选的对照改变了完成顺序，但未改善平均完成时间，因此默认实现不启用该规则。

10 讨论

FlowCausal 优化什么。 FlowCausal 面向这样一种处理器执行：工具进程已经阻塞，而某些子进程结束后才能解除等待。它不判断软件修改是否正确，也不向某个会话增加处理器。它识别能够解除已观察等待的存活进程，并把这些进程提前运行的时间计入所属会话。

与应用内部并行的关系。 构建系统和测试框架已经会决定自身进程数量与并行方式。FlowCausal 不修改其工作进程数，也不覆盖应用设置的亲和性。调度器只处理已经可以运行的进程；没有确认等待时，仍采用普通虚拟运行时间排序。

适用范围。 该机制依赖标准进程创建和等待行为，不依赖特定语言或智能体框架，因而能够覆盖 HarnessReplayBench 中的命令解释器、测试框架、编译器和工作进程池。eventfd、futex 或远程服务等其他完成方式需要分别验证，不能直接视为相同的等待关系。

部署方式。 `sched_ext` 允许在不修改内核核心调度代码的情况下加载策略，BPF 调度器退出后还会回到普通调度器 [20]。C 程序登记可以接入容器或微虚拟机的启动流程；会话权重、处理器掩码和普通控制组设置仍由系统管理者决定。

11 相关工作

相关系统分别提供可编程处理器调度、预先给出的工作流或者智能体编号。下面比较哪些系统能够给出会话边界，以及哪些系统能够观察本文调度所需的实时进程关系。

Linux 与应用处理器调度。 已有研究通过缺陷分析和策略重建考察 Linux 调度器 [21]。Nest 通过任务放置保持局部性 [18]；Caladan 将处理器分配与低时延运行时调度结合 [9]；Demeter 针对黑盒负载自适应调整处理器服务 [35]；其他工作分别研究虚拟处理器抽象、分组调度、考虑任务相互影响的放置，以及高性能计算任务的系统级调度 [3, 11, 14, 43]。早期依赖驱动调度根据系统调用历史和反馈结果推测资源请求者与提供者之间的关系 [42]。FlowCausal 则把运行时提供的工具调用编号与内核确认的进程等待结合，建立只属于当前工具调用的工具返回关系，并把调整次序所用的处理器时间计入所属会话。

可编程调度与内核自适应。 Arachne 面向应用线程进行处理器感知管理 [28]。ghOSt、Skyloft 和用户态进程调度器把策略判断移出传统内核代码 [13, 15, 40]；LibPreemptible 加快用户态抢占 [19]；Enoki 支持快速开发 Linux 调度器 [24]；`sched_ext` 则提供本文使用的 BPF 调度接口 [20]。Xkernel 和 AutoOS 调整或生成范围更广的内核策略 [6, 7]。调度器协作锁和应用提供信息的内核同步机制说明，应用结构可以支撑有针对性的内核动作 [26, 27]。FlowCausal 把运行时提供的工具调用编号与内核确认的等待事实结合，并把提前使用的处理器时间计入所属会话。

集群与工作流调度。 Decima 为数据处理依赖图学习调度策略 [22]。Twine 和 Gödel 分别在作业、容器和资源池层面管理共享基础设施 [34, 38]；Autopilot 调整工作负

载容量 [29]；Singularity 支持抢占与迁移 [31]；MLaaS 和 Beehive 分别处理加速器作业和高频小任务 [8, 36]。这些系统接收显式作业、依赖图或资源请求；FlowCausal 处理的是已经运行的沙箱会话内部不断出现的进程。

模型与加速器调度。 Orca 调度自回归生成的迭代 [41]；vLLM 通过 PagedAttention 管理键值缓存 [17]；Sarathi-Serve、DistServe 和 Llumnix 分别协调批处理、分离式服务和请求迁移 [2, 33, 45]。REEF 和 XSched 实现加速器抢占 [12, 30]；PowerInfer 在处理器与图形处理器之间划分推理计算 [32]；SAGA 与 Cortex 调度智能体的加速器工作流 [10, 25]。FlowCausal 调度的是执行代码库工具的主机进程。

智能体基准与运行时。 SWE-bench、SWE-agent、SWE-rebench 和 Terminal-Bench 使代码库与终端任务能够重复执行 [4, 16, 23, 39]。Firecracker 提供高效微虚拟机基础 [1]。AgentCgroup 管理工具调用资源 [44]，Crab 根据智能体进展安排检查点保存 [37]。HarnessReplayBench 使用这些公开任务环境在受控条件下比较主机调度器；FlowCausal 在登记后的会话内部确认这些接口没有表达的工具返回关系。

12 结论

本文提出 FlowCausal，将运行时提供的工具调用标识与内核确认的阻塞事件结合，恢复工具返回所依赖的进程。系统以 cgroup 组织会话，在会话份额内提供有上限的提前运行，并按实际 CPU 使用记账。HarnessReplayBench 的 50 个公开任务用于评价这一设计：两组四会话实验中，相对 Linux 的平均完成时间降低 4.2%–5.2%；构建等待消融中，工具返回关系相对仅会话调度进一步降低完成时间 4.88%。

在 4 个 CPU、8 个工作流、每种方案交错运行 3 轮的对照中，FlowCausal 相对原生 Linux 将平均完成时间降低 7.32%，全部完成时间增加 2.65%。跨命令依赖原型在固定工作量消融中降低平均完成时间 15.79%，但在完整工具回放中未取得额外收益。这些结果说明，实际等待关系可用于改善会话内的执行次序，其作用取决于等待进程的计算量、后台竞争及依赖转换是否发生；平均完成时间与全部完成时间仍需分别评估。

参考文献

[1] Alexandru Agache, Marc Brooker, Andreea Florescu, Alexandra Iordache, Anthony Liguori, Rolf Neugebauer, Phil Piwonka, and Diana-Maria Popa. Firecracker: Lightweight virtualization for serverless applications. In *17th USENIX Symposium on Networked Systems Design and Implementation (NSDI)*, 2020. URL `https://www.usenix.org/conference/nsdi20/presentation/agache`.

[2] Amey Agrawal, Nitin Kedia, Ashish Panwar, Jayashree Mohan, Nipun Kwatra, Bhargav S. Gulavani, Alexey Tumanov, and Ramachandran Ramjee. Taming throughput-latency tradeoff in LLM inference with Sarathi-Serve. In *18th USENIX Symposium on Operating Systems Design and Implementation (OSDI)*, 2024. URL `https://www.usenix.org/conference/osdi24/presentation/agrawal`.

[3] Daniel Alvarez et al. nOS-V: Co-executing HPC applications using system-wide task scheduling. In *IEEE International Parallel and Distributed Processing Symposium (IPDPS)*, pages 312–324, 2024.

[4] Igor Badertdinov, Alexander Golubev, Evgeny Andryunin, Vladimir Ostepyan, Denys Nahorna, Kirill Eremenko, Aleksey Kretov, Dmitry Kalashnikov, Ilya Makarov, Valerii Plekhanov, Dmitry Evdokimov, Pavel Shvechikov, and Aleksei Shpilman. SWE-rebench: An automated pipeline for task collection and decontaminated evaluation of software engineering agents. In *Advances in Neural Information Processing Systems: Datasets and Benchmarks Track*, 2025. URL `https://arxiv.org/abs/2505.20411`.

[5] Tingjia Cao, Shawn Wanxiang Zhong, Caeden Whitaker, Ke Han, Andrea C. Arpaci-Dusseau, and Remzi H. Arpaci-Dusseau. kSTEP: Characterization and deterministic testing of Linux CPU scheduler bugs. In *20th USENIX Symposium on Operating Systems Design and Implementation (OSDI)*, 2026. URL `https://www.usenix.org/conference/osdi26/presentation/cao`.

[6] Huilai Chen, Yuanbo Wen, Limin Cheng, Shouxu Kuang, Yumeng Liu, Weijia Li, Ling Li, Rui Zhang, Xinkai Song, Wei Li, Qi Guo, and Yunji Chen. AutoOS: Make your OS more powerful by exploiting large language models. In *Proceedings of the 41st International Conference on Machine Learning (ICML)*, 2024. URL `https://proceedings.mlr.press/v235/chen24at.html`.

[7] Zhongjie Chen, Wentao Zhang, Yulong Tang, Ran Shu, Fengyuan Ren, Tianyin Xu, and Jing Liu. Xkernel: Principled performance tunability of operating system kernels. In *20th USENIX Symposium on Operating Systems Design and Implementation (OSDI)*, 2026. URL `https://www.usenix.org/conference/osdi26/presentation/chen-zhongjie`.

[8] Yuxia Cheng, Linfeng Xu, Tongkai Yang, Wei Wu, Zhiqiang Lin, Antong Yu, and Wenzhi Chen. Beehive: Decentralised high-frequency small tasks scheduling in large clusters. *IEEE Transactions on Parallel and Distributed Systems*, 36(6):1326–1337, 2025. doi: 10.1109/TPDS.2025.3563457.

[9] Joshua Fried, Zhenyuan Ruan, Amy Ousterhout, and Adam Belay. Caladan: Mitigating interference at microsecond timescales. In *14th USENIX Symposium on Operating Systems Design and Implementation (OSDI)*, 2020. URL `https://www.usenix.org/conference/osdi20/presentation/fried`.

[10] Dongxin Guo, Jikun Wu, and Siu Ming Yiu. SAGA: Workflow-atomic scheduling for AI agent inference on GPU clusters. In *Proceedings of the 35th International Symposium on High-Performance Parallel and Distributed Computing (HPDC)*, 2026. URL `https://arxiv.org/abs/2605.00528`.

[11] Ennan Guo et al. Optimizing task scheduling in cloud VMs with accurate vCPU abstraction. In *Proceedings of the European Conference on Computer Systems (EuroSys)*, pages 753–768, 2025.

[12] Mingcong Han, Hanze Zhang, Rong Chen, and Haibo Chen. Microsecond-scale preemption for concurrent GPU-accelerated DNN inferences. In *16th USENIX Symposium on Operating Systems Design and Implementation (OSDI)*, 2022. URL `https://www.usenix.org/conference/osdi22/presentation/han`.

[13] Jack Tigar Humphries, Neel Natu, Ashwin Chaugule, Ofir Weisse, Barret Rhoden, Josh Don, Luigi Rizzo, Oleg Rombakh, Paul Turner, and Christos Kozyrakis. ghOSt: Fast and flexible user-space delegation of linux scheduling. In *Proceedings of the 28th ACM Symposium on Operating Systems Principles (SOSP)*, 2021. doi: 10.1145/3477132.3483542.

[14] Al Amjad Tawfiq Isstaif, Evangelia Kalyvianaki, and Richard Mortier. Mitigating context switching in densely packed Linux clusters with latency-aware group scheduling. arXiv preprint arXiv:2508.15703, 2025. URL `https://arxiv.org/abs/2508.15703`.

[15] Yifan Jia et al. Skyloft: A general high-efficient scheduling framework in user space. In *Proceedings of the 30th ACM Symposium on Operating Systems Principles (SOSP)*, pages 265–279, 2024.

[16] Carlos E. Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik Narasimhan. SWE-bench: Can language models resolve real-world github issues? In *International Conference on Learning Representations (ICLR)*, 2024. URL `https://openreview.net/forum?id=VTF8yNQM66`.

[17] Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph E. Gonzalez, Hao Zhang, and Ion Stoica. Efficient memory management for large language model serving with PagedAttention. In *Proceedings of the 29th ACM Symposium on Operating Systems Principles (SOSP)*, 2023. doi: 10.1145/3600006.3613165.

[18] Julia Lawall, Himadri Chhaya-Shailesh, Jean-Pierre Lozi, Baptiste Lepers, Willy Zwaenepoel, and Gilles Muller. OS scheduling with Nest: Keeping tasks close together on warm cores. In *Proceedings of the 17th European Conference on Computer Systems (EuroSys)*, 2022. doi: 10.1145/3492321.3519585.

[19] Yilong Li et al. Libpreemptible: Enabling fast, adaptive, and hardware-assisted user-space scheduling. In *IEEE International Symposium on High-Performance Computer Architecture (HPCA)*, pages 922–936, 2024.

[20] Linux Kernel Developers. Extensible scheduler class. Linux kernel documentation, 2026. URL `https://docs.kernel.org/scheduler/sched-ext.html`. Accessed July 2026.

[21] Jean-Pierre Lozi, Baptiste Lepers, Justin Funston, Fabien Gaud, Vivien Quéma, and Alexandra Fedorova. The Linux scheduler: A decade of wasted cores. In *Proceedings of the 11th European Conference on Computer Systems (EuroSys)*, 2016. doi: 10.1145/2901318.2901326.

[22] Hongzi Mao, Malte Schwarzkopf, Shaileshh Bojja Venkatakrishnan, Zili Meng, and Mohammad Alizadeh. Learning scheduling algorithms for data processing clusters. In *Proceedings of the ACM Special Interest Group on Data Communication (SIGCOMM)*, 2019. doi: 10.1145/3341302.3342080.

[23] Mike A. Merrill, Alexander G. Shaw, Nicholas Carlini, et al. Terminal-bench: Benchmarking agents on hard, realistic tasks in command line interfaces. arXiv preprint arXiv:2601.11868, 2026. URL https://arxiv.org/abs/2601.11868.

[24] Samantha Miller, Anirudh Kumar, Tanay Vakharia, Ang Chen, Danyang Zhuo, and Thomas Anderson. Enoki: High velocity Linux kernel scheduler development. In *Proceedings of the European Conference on Computer Systems (EuroSys)*, 2024.

[25] Nikos Pagonas, Yeounoh Chung, Kostis Kaffes, and Arvind Krishnamurthy. Cortex: Workflow-aware resource pooling and scheduling for agentic serving. arXiv preprint arXiv:2510.14126, 2025. URL https://arxiv.org/abs/2510.14126.

[26] SeongJae Park et al. Application-informed kernel synchronization primitives. In *16th USENIX Symposium on Operating Systems Design and Implementation (OSDI)*, pages 667–682, 2022.

[27] Yash Patel et al. Avoiding scheduler subversion using scheduler-cooperative locks. In *Proceedings of the European Conference on Computer Systems (EuroSys)*, pages 1–17, 2020.

[28] Henry Qin et al. Arachne: Core-aware thread management. In *13th USENIX Symposium on Operating Systems Design and Implementation (OSDI)*, pages 145–160, 2018.

[29] Krzysztof Rzadca et al. Autopilot: Workload autoscaling at Google. In *Proceedings of the European Conference on Computer Systems (EuroSys)*, pages 1–16, 2020.

[30] Weihang Shen, Mingcong Han, Jialong Liu, Rong Chen, and Haibo Chen. XSched: Preemptive scheduling for diverse XPUs. In *19th USENIX Symposium on Operating Systems Design and Implementation (OSDI)*, 2025. URL https://www.usenix.org/conference/osdi25/presentation/shen-weihang.

[31] Dharma Shukla et al. Singularity: Planet-scale, preemptive and elastic scheduling of AI workloads. arXiv preprint arXiv:2202.07848, 2022. URL https://arxiv.org/abs/2202.07848.

[32] Yixin Song, Zeyu Mi, Haotong Xie, and Haibo Chen. Powerinfer: Fast large language model serving with a consumer-grade GPU. In *Proceedings of the 30th ACM Symposium on Operating Systems Principles (SOSP)*, 2024. doi: 10.1145/3694715.3695964.

[33] Biao Sun, Ziming Huang, Hanyu Zhao, Wencong Xiao, Xinyi Zhang, Yong Li, and Wei Lin. Llumnix: Dynamic scheduling for large language model serving. In *18th USENIX Symposium on Operating Systems Design and Implementation (OSDI)*, 2024. URL https://www.usenix.org/conference/osdi24/presentation/sun-biao.

[34] Chunqiang Tang et al. Twine: A unified cluster management system for shared infrastructure. In *14th USENIX Symposium on Operating Systems Design and Implementation (OSDI)*, pages 787–803, 2020.

[35] Wei Tang et al. Demeter: QoS-aware CPU scheduling to reduce power consumption of multiple black-box workloads. In *Proceedings of the ACM Symposium on Cloud Computing (SoCC)*, pages 31–46, 2022.

[36] Qizhen Weng et al. MLaaS in the wild: Workload analysis and scheduling in large-scale heterogeneous GPU clusters. In *19th USENIX Symposium on Networked Systems Design and Implementation (NSDI)*, pages 945–960, 2022.

[37] Tianyuan Wu, Chaokun Chang, Lunxi Cao, Wei Gao, and Wei Wang. Crab: A semantics-aware checkpoint/restore runtime for agent sandboxes. arXiv preprint arXiv:2604.28138, 2026. URL https://arxiv.org/abs/2604.28138.

[38] Wei Xiang et al. Gödel: Unified large-scale resource management and scheduling at ByteDance. In *Proceedings of the ACM Symposium on Cloud Computing (SoCC)*, pages 308–323, 2023.

[39] John Yang, Carlos E. Jimenez, Alexander Wettig, Kilian Lieret, Shunyu Yao, Karthik Narasimhan, and Ofir Press. SWE-agent: Agent-computer interfaces enable automated software engineering. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2024.

[40] Kenichi Yasukata et al. Developing process scheduling policies in user space with common OS features. In *Asia-Pacific Workshop on Systems (APSys)*, pages 38–44, 2024.

[41] Gyeong-In Yu, Joo Seong Jeong, Geon-Woo Kim, Soojeong Kim, and Byung-Gon Chun. Orca: A distributed serving system for transformer-based generative models. In *16th USENIX Symposium on Operating Systems Design and Implementation (OSDI)*, 2022. URL https://www.usenix.org/conference/osdi22/presentation/yu.

[42] Haoqiang Zheng and Jason Nieh. SWAP: A scheduler with automatic process dependency detection. In *First Symposium on Networked Systems Design and Implementation (NSDI 04)*, San Francisco, CA, 2004. USENIX Association. URL https://www.usenix.org/conference/nsdi-04/swap-scheduler-automatic-process-dependency-detection.

[43] Yue Zheng et al. iAware: Interaction-aware task scheduling for reducing resource contention in mobile systems. *ACM Transactions on Embedded Computing Systems*, 2023.

[44] Yusheng Zheng, Jiakun Fan, Quanzhi Fu, Yiwei Yang, Wei Zhang, and Andi Quinn. AgentCgroup: Understanding and controlling OS resources of AI agents. arXiv preprint arXiv:2602.09345, 2026. URL https://arxiv.org/abs/2602.09345.

[45] Yinmin Zhong, Shengyu Liu, Junda Chen, Jianbo Hu, Yibo Zhu, Xuanzhe Liu, Xin Jin, and Hao Zhang. DistServe: Disaggregating prefill and decoding for goodput-optimized large language model serving. In *18th USENIX Symposium on Operating Systems Design and Implementation (OSDI)*, 2024. URL https://www.usenix.org/conference/osdi24/presentation/zhong-yinmin.